

ACCOUNTING FOR SIMULATOR BRANCHING IN PREDICTIVE REPRESENTATION LEARNING

Internal working draft v0 — not submitted to any venue

Anonymous authors

Paper under double-blind review

ABSTRACT

Action-conditional predictive objectives are a standard way to train history representations in partially observable environments, and a resettable simulator makes it tempting to supervise them with the outcomes of several alternative actions taken from the same saved state. Such branching consumes environment transitions and relies on a privilege, state restore, that ordinary interaction does not provide. We study how to account for this privilege when comparing representation learners. We implement a small aliased T-maze in which the current observation at a junction is identical for both values of an earlier clue, a transition ledger that charges every main-trajectory, branch and evaluation step against one hard cap and counts restores separately, and a fixed readout protocol that trains a fresh action-conditional reward head on a frozen encoder and scores a model-predictive planner by its junction decision (forced-commit accuracy). An evaluator diagnostic with fixed encoders shows that this readout can separate informative from uninformative histories: a hand-designed history-only control reaches forced-commit accuracy 1.000 in all three diagnostic seeds, while an untrained recurrent encoder and a memoryless control both stay at 0.500, leaving a headroom of 0.500. In the same runs, the full-action planning success of the two clue-blind encoders differs only through stalling behaviour, so we do not use it to compare representations. A 2×2 study that crosses action conditioning (an action-conditional predictor versus an action-free ablation) with the collection method (branching versus no branching) under an equal transition budget has been specified but has not been run at the time of this draft (v0). We make no claim that low prediction error guarantees planning for arbitrary policies, and all results come from one synthetic task family with three seeds.

1 INTRODUCTION

In a partially observable environment the current observation can be identical in situations that call for different actions. This is perceptual aliasing (Whitehead & Ballard, 1991; McCallum, 1996). An agent must then act on a summary of its history. Predictive state representations make the point precise: a history is summarised by predictions of *action-conditional* tests, that is, of future observations given future actions (Littman et al., 2001; Singh et al., 2004). Neural descendants of this idea train recurrent history encoders with multi-step, action-conditional prediction losses (Gregor et al., 2019; Guo et al., 2018; 2020; Kwon et al., 2024). Related latent-consistency objectives with action-conditional dynamics are standard in model-based reinforcement learning (Schwarzer et al., 2021; Hansen et al., 2024; Schrittwieser et al., 2020).

An action-conditional target is only observed for the actions that were actually taken. With a resettable simulator one can do more: save the complete simulator state, try several first actions from it, and restore the state between attempts. This *branching* yields paired counterfactual targets from the same history. It is an old device, used for instance by the vine procedure of TRPO (Schulman et al., 2015) and by PSR learning with resets (James & Singh, 2004). Branching is not free, however. Each branch step is an environment transition, and the restore operation is a privilege that ordinary interaction with a real system does not provide. Comparisons of representation learners that ignore either

cost can credit an objective with gains that actually come from extra experience or from simulator access.

This draft asks a narrow question: *under an equal environment-transition budget and a fixed readout protocol, how do the effects of branching collection and of action conditioning separate?* We do not propose a new objective. Action-conditional prediction, multi-step prediction, recurrent world models, and state-restore branching are all prior work (Section 2). Our contributions are limited to the following.

- **An accounting protocol and implementation.** A single transition ledger charges every main-trajectory, branch, and evaluation step against one hard cap. Snapshots and restores are counted separately and are forbidden during evaluation. Actors and learners receive only the observable history, never simulator state or evaluator labels. These properties are checked by unit tests. The tests are technical checks, not evidence about any method.
- **An evaluator diagnostic.** Before comparing learned encoders, we check that the read-out (a frozen encoder, a fresh action-conditional reward head, and an exhaustive model-predictive planner) can separate informative from uninformative representations. It can: a hand-designed history-only control and an untrained recurrent encoder differ by 0.500 in forced-commit accuracy in every diagnostic seed (Section 5.1).
- **A small controlled 2×2 study** that crosses action conditioning with branching under an equal transition budget and reports experience and compute separately (Section 5.2).

We also report what the evidence does not show. The primary metric, forced-commit accuracy, measures a single junction decision, not planning success in general. The full-action planner’s success rate is confounded by stalling behaviour. All results come from one synthetic task family with three seeds, and the comparison with an external action-conditioned method such as that of Kwon et al. (2024) is not yet run.

2 RELATED WORK

Predictive state and history representations. PSRs represent state as predictions of action-conditional tests (Littman et al., 2001; Singh et al., 2004). Temporal-difference networks learn such predictions from the actions actually taken (Sutton & Tanner, 2004). Predictive-state decoders and PSRNNs attach future-observation prediction to recurrent networks; in the formulations we checked, the targets are not conditioned on future actions (Venkatraman et al., 2017; Downey et al., 2017), which makes them structurally closest to our action-free ablation. The approximate information state framework bounds the planning loss of a history compression that predicts rewards and its own next value given an action (Subramanian et al., 2022). Ni et al. (2024) relate one-step self-predictive history objectives to that framework.

Action-conditional multi-step prediction. SimCore rolls a recurrent belief forward conditioned only on the agent’s actions (Gregor et al., 2019). CPC|Action and PBL use action-conditional multi-step predictions from compressed histories (Guo et al., 2018; 2020). Guo et al. (2018) report that multi-step prediction and action conditioning are critical for their belief representations; our action-conditional versus action-free contrast is therefore not a new question. Multi-step latent prediction conditioned on actions also underlies SPR (Schwarzer et al., 2021), TD-MPC2 (Hansen et al., 2024), MuZero (Schrittwieser et al., 2020), and earlier action-conditional video prediction (Oh et al., 2015). Recurrent world models such as PlaNet and DreamerV3 learn latent dynamics in POMDPs (Hafner et al., 2019; 2023). Khetarpal et al. (2025) compare per-action and policy-marginal self-prediction theoretically in MDPs. Task-oriented alternatives include bisimulation metrics (Zhang et al., 2021), value equivalence (Grimm et al., 2020), and successor features (Dayan, 1993; Barreto et al., 2017).

Closest work. Kwon et al. (2024) train a history encoder with an action-conditional k -step observation predictor, decouple representation learning from reinforcement learning, and evaluate on memory tasks. Their test actions are inserted once per episode along the executed trajectory, without simulator branching. In the version we read, we did not find an action-free ablation. Our action-conditional variant is a small re-implementation in the same spirit, not an official reproduction of their method.

Resets, branching, and memory tasks. State restore for branching rollouts appears in TRPO’s vine procedure (Schulman et al., 2015). Reset-and-replay is used to estimate PSR tests from a common history (James & Singh, 2004), and local simulator access has been analysed theoretically (Mhammedi et al., 2024). In our deterministic task, reset-and-replay and save-and-restore are nearly equivalent. T-maze memory tasks and recurrent or world-model baselines for them are studied by Ni et al. (2022), Ni et al. (2023), and Samsami et al. (2024).

3 PROBLEM AND EXPERIMENTAL SETTING

Aliased T-maze. An episode starts in a clue cell whose observation reveals a hidden binary clue $c \in \{A, B\}$. The agent then walks a corridor of length L , sampled uniformly from $\{3, \dots, 6\}$. Corridor observations are drawn from 4 distractor symbols independently of c . The corridor ends at a junction whose observation is identical for both clues. The actions are FORWARD, LEFT, and RIGHT. In the corridor, LEFT and RIGHT leave the agent in place; at the junction, FORWARD leaves the agent in place, and LEFT or RIGHT ends the episode in an arm. In the symmetric variant used throughout, the arm that matches the clue gives a goal observation and reward $+1$, and the other arm gives reward -1 . Episodes time out after $L + 2 + 6$ steps. Two histories that differ only in the clue therefore share the current observation at the junction, yet the consequence of LEFT differs between them. Under any clue-independent distribution over action sequences, the action-marginal distribution of future observations and rewards is identical for both clues. Our tests confirm this by exact enumeration for horizons up to three, starting at the junction and one step before it, and also confirm that the action-conditional futures differ. This is a direct instance of the PSR observation that the relevant predictions are action-conditional (Littman et al., 2001; Singh et al., 2004), and we do not present it as a new result.

Information access. The simulator’s complete state is private: clue, corridor length, position, and random-number-generator state. It is available only to data collectors (for snapshot and restore) and to the evaluator (for labels). Actors, encoders, prediction heads, and the planner receive only the observable history $h_t = (o_0, a_0, r_1, \dots, a_{t-1}, r_t, o_t)$. Ground-truth labels are returned in a separate structure that training code never receives. Unit tests check the relevant function signatures, the whitelisted record fields, that serialized datasets contain no hidden fields, and that the encoder output at time t is unaffected by later inputs.

Transition budget and the restore privilege. Every environment step, whether on the main trajectory, in a branch rollout, or during evaluation, is charged to a ledger *before* it executes. A step that would exceed the cap is not executed. Data collection spends the budget exactly, so arms with equal caps also have equal *actual* transition spend. Snapshots and restores are not transitions, but they are a simulator privilege; the ledger counts them separately, and the evaluation ledger forbids them. An equal transition budget does not imply equal compute, and we report arithmetic cost (multiply-accumulates, MAC) and CPU time separately.

Branching collection. Main trajectories follow a history-based behaviour policy. It is uniform at the junction and presses FORWARD with probability 0.8 elsewhere. At each main step, with probability 0.25, the collector snapshots the simulator state. For each of the three first actions it then rolls out up to $k = 3$ steps with a uniform continuation policy, restoring the snapshot between rollouts and after the last one. Random streams are separated so that branching never changes the main trajectory; a test checks this property. A branch set is started only if the remaining budget can cover it.

Readout protocol. Representations are compared through one fixed downstream path. The encoder is frozen. Its features are z-scored per dimension, with statistics fitted only on the readout training data. A fresh action-conditional reward head, a latent rollout with hidden size 16, is trained with squared reward error at learning rate 0.01. The head initialization, data, and optimizer are identical across encoders. We keep the checkpoints after 20/40/80 epochs of a single run and select the one with the lowest development reward error; the test split is never used for selection. A model-predictive planner enumerates all action sequences of length 2 and scores them by discounted predicted reward ($\gamma = 0.9$).

Metrics. Each test episode starts with a scripted, observation-only prefix that presses FORWARD until the junction observation appears. *Forced-commit accuracy*, the primary metric, is the fraction of test episodes in which the planner, restricted to first actions LEFT and RIGHT at the first junction arrival, prefers the optimal arm. It measures one aliased decision and is not a planning success rate. The *full-action* planner then acts with all three actions until the episode ends. From it we report success, mean return, and timeout rate, together with the number of exact forced-commit ties. The evaluator assigns clues by alternating episode index, so every constant decision rule scores exactly 0.5. A logistic probe of the clue on the junction features is fitted on a separate calibration split. We use it only as a diagnostic: failure of a linear probe does not show that the features contain no clue information.

4 VARIANTS UNDER STUDY

All learned encoders are Elman recurrent networks with tanh units and hidden size 16. The input at time t is the one-hot observation o_t , the one-hot previous action, and the previous reward, and the recurrence is $z_t = \tanh(W[x_t; z_{t-1}] + b)$ with $z_{-1} = 0$. The predictor is a latent rollout $u_0 = z_t$, $u_{j+1} = \tanh(W_d[u_j; \alpha_j] + b_d)$, with linear heads for observation logits and reward. Pretraining minimizes the cross-entropy of o_{t+j+1} plus the squared error of r_{t+j+1} for $j < k$. The loss is taken over all k -step windows of the main trajectories and over all branch rollouts. After a termination, targets are an absorbing padding symbol with reward zero and remain supervised, because the end of an episode is observable. After a truncation (timeout or budget), targets are masked. The variants differ only in α_j and in the pretraining data.

- **ACP (action-conditional prediction):** α_j is the one-hot action a_{t+j} . The objective fits $p(o_{t+j+1}, r_{t+j+1} \mid h_t, a_{t:t+j})$. This conditional target is unconfounded only if the actions in the data do not depend on hidden state. Here the behaviour policy reads only the history, and the continuation policy is uniform.
- **MSP (action-free multi-step prediction):** $\alpha_j = 0$. The objective fits the action-marginal future under the action distribution of the data, so it depends on the behaviour policy. MSP is an *ablation* of ACP. It has the same nominal parameter count, but the action columns of W_d receive no gradient, and we do not treat it as a strong or state-of-the-art baseline.
- **Collection:** *branching* data or *no-branching* data under the same transition cap. The difference between them is a difference in collection method, not a controlled comparison on identical data.
- **Reference encoders:** an *untrained* recurrent encoder with the same initialization as the learned encoders; a hand-designed *history-only informative control* whose features are the clue identity read from o_0 together with the one-hot current observation; and, in the diagnostic only, a *memoryless control* that sees only the one-hot current observation. The informative control is a reference point for the readout, not an upper bound on what a learned encoder can achieve.

Low prediction error is not claimed to guarantee planning performance. The readout and planner above cover one decision, horizon 2, and the action support present in the data. Nothing here speaks to other policies, longer horizons, or states outside the data’s coverage.

5 EXPERIMENTS AND OBSERVATIONS

5.1 EVALUATOR DIAGNOSTIC WITH FIXED ENCODERS

Question and design. Before comparing learned encoders, we asked whether the readout of §3 can separate informative from uninformative histories at all. If it cannot, a learned encoder cannot be credited with anything. Three fixed encoders went through the identical readout: the history-only informative control, the untrained recurrent encoder, and the memoryless control. The diagnostic used 10,000 readout-training transitions (collected with branching), 2,000 development transitions, 200 calibration episodes, and 200 clue-balanced test episodes per seed. Seeds 1000, 1001, 1002 were reserved for this diagnostic. The decision rules were fixed before the run. The evaluator is unresolved if the informative control falls below 0.90 or the memoryless control deviates from 0.5

Table 1: Evaluator diagnostic (fixed encoders, symmetric maze). Entries are per seed (1000, 1001, 1002). Forced-commit is the primary junction-decision metric. Full-action success and timeout come from the unrestricted planner. “Epochs” is the head checkpoint selected on development reward error. Source: `runs/diag_20260926T160006Z_281020c`.

Encoder	Forced-commit	Full-action success	Timeout	Probe (test)	Epochs
History-only informative control	1.000 / 1.000 / 1.000	1.000 / 1.000 / 1.000	0.000 / 0.000 / 0.000	1.00 / 1.00 / 1.00	80 / 80 / 40
Untrained recurrent encoder	0.500 / 0.500 / 0.500	0.500 / 0.500 / 0.500	0.000 / 0.000 / 0.000	0.47 / 0.53 / 0.57	20 / 20 / 80
Memoryless control	0.500 / 0.500 / 0.500	0.500 / 0.000 / 0.000	0.000 / 1.000 / 1.000	0.50 / 0.50 / 0.50	80 / 80 / 80

by more than 0.10 in any seed. There is headroom if the informative control exceeds the untrained encoder by at least the minimum effect of interest (0.10) in every seed.

Observations. Table 1 lists the per-seed results. All 9 of 9 cells completed, and the verdict is HEADROOM_PRESENT. The informative control reached forced-commit accuracy 1.000 in every seed. The untrained recurrent encoder and the memoryless control both stayed at exactly 0.500. Each chose the same arm in every test episode, which on a clue-balanced test set gives exactly one half. A leak of episode index or of the balancing pattern into the inputs would have shown up as above-chance accuracy for these clue-blind encoders, and it did not. The linear clue probe on the untrained encoder’s junction features scored between 0.47 and 0.57 on test episodes. The readout could not extract the clue from these features, but that does not show that the features carry no clue information.

The full-action metric is confounded. Full-action success separates the two clue-blind encoders, for which forced-commit accuracy is identical. The untrained encoder committed to an arm (success 0.500). The memoryless control, in two seeds, preferred FORWARD at the junction until timeout (timeout rate up to 1.000). The difference reflects commit-versus-stall behaviour of a model-predictive planner with an uninformative head, not the clue content of the representation. We therefore report full-action metrics but do not base comparisons on them.

Cost. The diagnostic used 53,503 environment transitions in total. Of these, 10,103 main and 19,897 branch steps were readout-training data, with 9,888 restores. Test episodes used 14,217 transitions and no restores. The run took 1,189 CPU seconds and 13.17 GMAC in a single process with peak resident memory 26.9 MB (commit 281020c). The readout-training data in this diagnostic were collected with branching, so this readout was *not* restore-free. The 2×2 study below uses a separate restore-free readout pool.

5.2 2×2 STUDY: ACTION CONDITIONING $\times$ BRANCHING

Status in this draft (v0). The study has been specified but not yet run. Its design is as follows. Six arms are evaluated with three seeds in the symmetric maze: ACP with branching, ACP without branching, MSP with branching, MSP without branching, the untrained recurrent encoder, and the history-only informative control. Each collection method gets a pretraining budget of 10,000 environment transitions per seed. The two branching arms share one dataset exactly, and so do the two no-branching arms. All six arms use the readout protocol above with a separate, shared, restore-free readout pool. The paired contrasts, the interaction, and the decision rules are listed in Appendix C. No results of this study are reported in v0.

6 DISCUSSION AND LIMITATIONS

What the current evidence supports. The accounting and information-access properties hold in our implementation, as checked by unit tests. These are technical checks, not evidence about any method. The fixed-encoder diagnostic shows that the readout can separate an informative history summary from uninformative ones by a wide margin in this task. It also shows that, at the diagnostic settings, an untrained recurrent encoder does not already solve the junction decision. This removes one reason a comparison of learned encoders could be uninformative. It does not show that any learned encoder fills the gap.

Limitations.

- **One synthetic task family.** All results come from a small deterministic aliased T-maze. The extreme values of 0.5 and 1.0 reflect this task and should not be expected elsewhere.
- **Reset privilege.** Branching requires a simulator that can save and restore arbitrary states. Any advantage of branching collection is conditional on that privilege, and it does not transfer to systems that lack it.
- **Three seeds.** Each comparison uses three seeds of one configuration. These are repetitions under a fixed design, not independent replications, and a non-significant or inconsistent contrast is not evidence of equivalence.
- **Forced-commit primary metric.** The primary metric scores one aliased decision under a scripted prefix. It is not a planning success rate, and we make no claim about other policies, horizons, or states outside the data’s coverage.
- **Head-dependent readout.** Conclusions are conditional on the frozen-encoder readout: feature standardization, a small reward head, three epoch candidates, and a depth-2 planner. A different readout could rank encoders differently. The probe is linear, and its failure is not a proof of absent information.
- **Weak comparison set.** The action-free ablation is not a strong baseline. We have not compared against an external action-conditioned method, for example the approach of Kwon et al. (2024), or against recurrent world models (Hafner et al., 2023; Samsami et al., 2024).
- **Protocol changes after early looks.** Several protocol choices were revised after smoke runs and before any comparison of learned encoders: clue-balanced test episodes, probe feature standardization, forced-commit as the primary metric, and longer head training. Appendix B lists these changes.

7 CONCLUSION

Comparisons of predictive representation learners that use simulator branching should charge every branch step to the same transition budget as ordinary interaction. They should count restores as a separate privilege, and they should report compute separately from experience. We implemented such an accounting in a small aliased T-maze, together with a fixed frozen-encoder readout. We showed that this readout discriminates a history-only informative control from an untrained recurrent encoder and a memoryless control, and that the full-action success metric is confounded by stalling. These results establish only that the evaluation can detect differences in this task. Whether branching collection or action conditioning provides such differences under an equal budget is the question of the 2×2 study, whose status is reported in §5.2.

AI USE STATEMENT

An AI coding agent (Claude Code, Anthropic) carried out most of the work under instructions from the human project owner. It wrote the environment, accounting, training, evaluation, and export code and the unit tests, it ran all experiments reported here, and it drafted this manuscript. An AI sub-agent used web search and page fetching to check bibliographic metadata and passages of the cited papers; each reference’s verification level is recorded in `paper/claim_evidence.tsv`. **Status: HUMAN REVIEW PENDING.** At the time of this draft, no human author has reviewed the code, the results, or the text, and the draft must not be treated as reviewed or endorsed.

REPRODUCIBILITY STATEMENT

Every number in this draft is produced by `paper/export_results.py` from the raw run files listed in `paper/result_sources.json`; the export manifest records their SHA-256 hashes. Each run directory contains its frozen configuration, a raw event log, per-cell results, and a manifest with the git commit, source and configuration hashes, transition ledgers, CPU time, multiply-accumulate counts, and peak memory. The implementation uses only the Python standard library. Appendix A gives the exact commands. The code and logs have not been publicly released.

REFERENCES

- André Barreto, Will Dabney, Rémi Munos, et al. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems 30*, 2017.
- Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993.
- Carlton Downey, Ahmed Hefny, Byron Boots, Geoffrey J. Gordon, and Boyue Li. Predictive state recurrent neural networks. In *Advances in Neural Information Processing Systems 30*, 2017.
- Karol Gregor, Danilo Jimenez Rezende, Frederic Besse, Yan Wu, Hamza Merzic, and Aäron van den Oord. Shaping belief states with generative environment models for RL. In *Advances in Neural Information Processing Systems 32*, 2019.
- Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems 33*, 2020.
- Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Bernardo A. Pires, and Rémi Munos. Neural predictive belief representations. *arXiv preprint arXiv:1811.06407*, 2018.
- Zhaohan Daniel Guo, Bernardo Avila Pires, Bilal Piot, et al. Bootstrap latent-predictive representations for multitask reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pp. 3875–3886, 2020.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, et al. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pp. 2555–2565, 2019.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Michael R. James and Satinder Singh. Learning and discovery of predictive state representations in dynamical systems with reset. In *Proceedings of the 21st International Conference on Machine Learning (ICML)*, 2004.
- Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, et al. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 258 of *PMLR*, pp. 181–189, 2025.
- Jeongyeol Kwon, Liu Yang, Robert Nowak, and Josiah Hanna. An empirical study on the power of future prediction in partially observable environments. *arXiv preprint arXiv:2402.07102*, 2024. Version 2, March 2025.
- Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems 14*, 2001.
- Andrew K. McCallum. *Reinforcement Learning with Selective Perception and Hidden State*. PhD thesis, University of Rochester, 1996.
- Zakaria Mhammedi, Dylan J. Foster, and Alexander Rakhlin. The power of resets in online reinforcement learning. In *Advances in Neural Information Processing Systems 37*, 2024.
- Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free RL can be a strong baseline for many POMDPs. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.
- Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in RL? decoupling memory from credit assignment. In *Advances in Neural Information Processing Systems 36*, 2023.

- Tianwei Ni, Benjamin Eysenbach, Erfan Seyedsalehi, et al. Bridging state and history representations: Understanding self-predictive RL. In *International Conference on Learning Representations (ICLR)*, 2024.
- Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard Lewis, and Satinder Singh. Action-conditional video prediction using deep networks in Atari games. In *Advances in Neural Information Processing Systems* 28, 2015.
- Mohammad Reza Samsami, Artem Zhohus, Janarthanan Rajendran, and Sarath Chandar. Mastering memory tasks with world models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.
- Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations (ICLR)*, 2021.
- Satinder Singh, Michael R. James, and Matthew R. Rudary. Predictive state representations: A new theory for modeling dynamical systems. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 512–519, 2004.
- Jayakumar Subramanian, Amit Sinha, Raihan Seraj, and Aditya Mahajan. Approximate information state for approximate planning and reinforcement learning in partially observed systems. *Journal of Machine Learning Research*, 23(12):1–83, 2022.
- Richard S. Sutton and Brian Tanner. Temporal-difference networks. In *Advances in Neural Information Processing Systems* 17, 2004.
- Arun Venkatraman, Nicholas Rhinehart, Wen Sun, et al. Predictive-state decoders: Encoding the future into recurrent networks. In *Advances in Neural Information Processing Systems* 30, 2017.
- Steven D. Whitehead and Dana H. Ballard. Learning to perceive and act by trial and error. *Machine Learning*, 7:45–83, 1991.
- Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *International Conference on Learning Representations (ICLR)*, 2021.

A REPRODUCTION COMMANDS AND TECHNICAL CHECKS

All code is pure Python 3.11 with no third-party packages. The commands below reproduce the reported runs, starting from the repository root.

```
python3 -m unittest discover -s tests -t . -v
OMP_NUM_THREADS=1 python3 -m acpr.diagnose \
  --config configs/diag_eval_discriminability.json --out runs/<id>
python3 paper/export_results.py
```

The command for the 2×2 study will be added once that study has been run. The unit tests check the following technical properties. First, the aliasing construction, verified by exact enumeration. Second, transition accounting: the recorded main and branch steps equal the ledger totals, the cap is hard and spent exactly, and restores are counted. Third, information access: signatures, field whitelists, serialized datasets, and encoder causality. Fourth, split disjointness. Fifth, horizon semantics: termination padding versus truncation masking. Sixth, finite-difference gradient checks of the pure-Python autodiff. Seventh, positive and negative controls of the readout, and CPU-cap enforcement through `RLIMIT_CPU`. These are numeric fixture checks of the implementation. They are not proofs and not evidence of method efficacy.

B HISTORY OF PROTOCOL CHANGES

The protocol changed during development, and we record the changes here for transparency. After the first smoke run, the clue probe collapsed to the majority class, and planner metrics were dominated by the test set’s clue base rate. The probe features were therefore standardized, and test clues were balanced by the evaluator. After the second smoke run and a positive-control check, forced-commit accuracy replaced full-action success as the primary metric. The reason was that an under-trained head made the full-action planner stall at the junction. For the same reason, head training was lengthened. The evaluator diagnostic then added train-only standardization of encoder features before the head, applied identically to every encoder, and checkpoint selection among 20/40/80 epochs. None of these changes was made after observing learned-encoder comparisons. They were, however, made after looking at smoke and control outputs, so seeds used in those runs count as development data.

C PLANNED EXPERIMENTS (NOT RUN)

None of the following has been run. No result of any of them appears in this draft.

- **2×2 study (specified in v0).** The arms are ACP and MSP, each with and without branching, plus an untrained recurrent encoder and the history-only informative control, with three seeds in the symmetric maze. The paired contrasts per seed are: $ACP_b - ACP_n$, $ACP_b - MSP_b$, $ACP_n - MSP_n$, each learned arm minus the untrained encoder, and the interaction $(ACP_b - MSP_b) - (ACP_n - MSP_n)$. The decision labels are EVALUATOR_UNRESOLVED, INCOMPLETE, SATURATED_COMPARISON, LEARNING_BENEFIT_NOT_SUPPORTED, and PRELIMINARY_COLLECTION_BENEFIT, together with an action-conditioning-only outcome that cannot be distinguished from the existing explanation.
- **Pre-registered pilot.** Ten seeds in both maze variants with a paired bootstrap analysis. It was not approved and not run.
- **Asymmetric control variant.** In this variant, action-marginal futures also carry the clue. It was not run.
- **Unstandardized readout.** The head without feature standardization, as in the original pilot plan. It was not run.
- **External comparisons.** An action-conditioned future-prediction method in the style of Kwon et al. (2024), and recurrent world models. Neither was run.
- **Other environments.** Longer corridors, other task families, and partially observable continuous control. None was run, and none is planned without a positive result here.